# Policy Optimization

Thang Doi

April 5, 2026

# Outline


1. Policy Gradient Theorem
2. Reduction Variance
3. Actor and Critic
4. Generalized Advantage Estimation (GAE)
5. Natural Policy Gradient Methods
6. Trust Region Policy Gradient
7. Proximal Policy Optimization
8. Off-policy Learning
9. Deterministic Policy Gradient
10. Reinforcement Learning as Probabilistic Inference
11. Maximum a Posteriori Policy Optimisation
12. Soft Actor-critic (SAC)

# Policy Gradient Theorem

# Advantage of Policy-based Learning

- Can learn a stochastic policy, instead of a deterministic policy. Because the world is complicated, the dynamic model changes over time.
- Naturally represents stochastic policies $\pi_\theta(a|s)$, which can improve exploration.
- An approximation policy can deal with a continuous action space.

# Policy Objective

- Consider the objective of the policy-based learning in the discounted MDP.

$$E_{\tau \sim p_\theta(\tau)}[\sum_t^T \gamma^t r(s_t, a_t)]$$

where

$$p_\theta(\tau) = p(s_0) \prod_{t=0}^{T} \pi(a_t|s_t) p(s_{t+1}|s_t, a_t)$$

with $p(s_0)$ is a prior distribution of the initial state.

- In RL, several problems are in a continual task with an infinite horizon, but we often limit the horizon step and the discounted objective instead of the average objective.

# Policy Gradient Theorem

- Policy-based reinforcement learning becomes an optimization problem.
- How to estimate the policy objective without depending on the dynamic transition model $p(s_{t+1}|a_t, a_t)$ and prior initial state $p(s_0)$.

**Theorem (Policy Gradient)**

Let $\pi_\theta(a|s)$ be a differentiable policy on action state space $A \times S$. And $p_\Theta(\tau)$ is a probability distribution over a finite $(S, A, R, \gamma|s_0)$ MDP with initial state $s_0$. Then, the policy gradient over a trajectory $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, ..., s_{T-1}, a_{T-1}, r_{T-1}, s_T)$ with objective $J(\theta) = E_{\tau \sim p_\Theta(\tau)}[G_\tau]$.

$$\nabla_\theta J(\theta) = E_{\tau \sim p_\Theta(\tau), \pi_\theta}[\sum_{t=0}^{T} \gamma^t Q_{\pi_\theta}(s_t, a_t) \nabla_\theta \log \pi_\theta(a_t|s_t)]$$

# Proof theorem

$$
\begin{aligned}
J(\theta) &= E_{\tau\sim p_\Theta}[G(\tau)] && (1)\\
&= \int_{\tau\in S\times A\times R} p_\Theta(\tau)G(\tau)d_\tau && (2)\\
\nabla_\theta J(\theta) &= \int_{\tau\in S\times A\times R} \nabla_\theta p_\Theta(\tau)G(\tau)d_\tau && (3)\\
&= \int_{\tau\in S\times A\times R} p_\Theta(\tau)\frac{\nabla_\theta p_\Theta(\tau)}{p_\Theta(\tau)}G(\tau)d_\tau && (4)\\
&= E_{\tau\sim p_\Theta}[\nabla_\theta \log(p_\Theta(\tau))G(\tau)] && (5)\\
&= E_{\tau\sim\Theta}[\nabla_\theta \log(p(s_0)\prod_{t=0}^{T}\pi(a_t|s_t)p(s_{t+1}|s_t,a_t))G(\tau)] && (6)\\
&= E_{\tau\sim\Theta}[\nabla_\theta(\log p(s_0)+\sum_{t=0}^{T}\log\pi_\theta(a_t|s_t)+\sum_{t=0}^{T}\log p(s_{t+1}|s_t,a_t))G(\tau && (7)
\end{aligned}
$$

# Proof Theorem

$$\nabla_\theta J(\theta) = E_{\tau\sim p_\Theta}[G(\tau)\sum_{t=0}^{T}\nabla_\theta \log \pi_\theta(a_t|s_t)] \tag{8}$$

$$= E_{\tau\sim p_\Theta}[\sum_{t=0}^{T} G(\tau)\nabla_\theta \log \pi_\theta(a_t|s_t)] \tag{9}$$

$$= E_{\tau\sim p_\Theta}[\sum_{t=0}^{T}(\sum_{k=0}^{T}\gamma^k r_{k+1})\nabla_\theta \log \pi_\theta(a_t|s_t)] \tag{10}$$

$$= E_{\tau\sim p_\Theta}[\sum_{t=0}^{T}(\sum_{k=0}^{t-1}\gamma^k r_{k+1} + \gamma^t\sum_{k=t}^{T}\gamma^{k-t} r_{k+1})\nabla_\theta \log \pi_\theta(a_t|s_t)] \tag{11}$$

# Proof Theorem

$$= E_{\tau\sim p_\Theta}[\sum_{t=0}^{T}(\underbrace{\sum_{k=0}^{t-1}\gamma^k r_{k+1}\nabla_\theta \log \pi_\theta(a_t|s_t)}_{\text{history trajactory}} + \gamma^t \sum_{k=t}^{T}\gamma^{k-t} r_{k+1}\nabla_\theta \log \pi_\theta(a_t|s_t))] \tag{12}$$

- At time $t$, the history trajectory with reward at step $k$ is independent of future actions. Therefore, we can delete these terms.

$$\nabla_\theta J(\theta) = E_{\tau\sim p_\Theta}[\sum_{t=0}^{T}\gamma^t G_t \nabla_\theta \log \pi_\theta(a_t|s_t)]$$

# Reinforce Algorithm

- By using Monte Carlo methods to sample entire episodes to compute the sample returns $G_t = \sum_{t=0}^{T} \gamma^t r_{t+1}$. The sample policy gradient follows Reinforce:

$$\nabla_\theta \hat{J}(\theta) = \gamma^t G_t \nabla \log \pi_\theta(a_t|s_t)$$

- Update policy parameter:

$$\Delta_{\theta t} = \alpha \gamma^t G_t \nabla_\theta \log \pi_\theta(a_t|s_t)$$

- However, we often ignore the term $\gamma^t$:

$$\Delta \theta_t = \alpha G_t \nabla_\theta \log \pi_\theta(a_t|s_t)$$

# Reinforce Pesudo-code Algorithm

**REINFORCE: Monte-Carlo Policy-Gradient Control (episodic) for $\pi_*$**

Input: a differentiable policy parameterization $\pi(a|s, \boldsymbol{\theta})$
Algorithm parameter: step size $\alpha > 0$
Initialize policy parameter $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):
  Generate an episode $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot|\cdot, \boldsymbol{\theta})$
  Loop for each step of the episode $t = 0, 1, \ldots, T-1$:
    $G \leftarrow \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k$ $\qquad (G_t)$
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha \gamma^t G \nabla \ln \pi(A_t|S_t, \boldsymbol{\theta})$

- Disadvantage of Reinforce:
  - Reinforce rising variance with quandratic $Q(s, a)$ value

$$Var[(\nabla_\theta \hat{J}(\theta) - \nabla_\theta J(\theta))^2] = E[(\nabla_\theta \log \pi_\theta(a|s) Q(s, a))^2] - (\nabla_\theta J(\theta))^2$$

  - Unstable learning

# Reduction Variance

# Reduce variance with a baseline

- We can reduce the variance of policy gradient by a baseline, but it does not lead to bias

$$\nabla_\theta \hat{J}(\theta) = \nabla_\theta \log \pi_\theta(a|s)[Q(s,a) - b(s)]$$

- $b(s)$ is a baseline estimate state-value.

# Reduce variance with a baseline

- we show that $E[\nabla_\theta \hat{J}(\theta)] = \nabla_\theta J(\theta)$ (unbias)

$$E[\nabla_\theta \hat{J}(\theta)] = E[\nabla_\theta \log \pi_\theta(a|s) Q(s,a)] - E[\nabla_\theta \log \pi_\theta(a|s) b(s)]$$

- let $A = E[\nabla_\theta \log \pi_\theta(a|s) b(s)]$, compute the expectation by integrate on state and action domain

$$\begin{aligned} A &= \int_{s \in S} p^\pi(s) b(s) \int_{a \in A} \pi_\theta(a|s) \nabla_\theta \log \pi_\theta(a|s) d_a d_s \\ &= \int_{s \in S} p^\pi(s) b(s) \int_{a \in A} \nabla_\theta \pi_\theta(a|s) d_a d_s \\ &= \int_{s \in S} p^\pi(s) b(s) \nabla_\theta (\int_{a \in A} \pi_\theta(a|s) d_a) d_s \\ &= \int_{s \in S} p^\pi(s) b(s) \nabla_\theta (1) ds = 0 \end{aligned}$$

- $(Q(s,a) - b(s))$ often called advantage function.

# Actor and Critic

# Optimal baseline

- Recall: previously introduced a reduced method by using a baseline value.
- Which baseline is the base to reduce the highest variance?. We can solve the optimization problem.

$$
\begin{aligned}
&\arg\min_{b(s)} Var_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a|s)(Q(s,a) - b(s))] \\
&= \arg\min_{b(s)} E_{\pi_\theta}[(\nabla_\theta \log \pi_\theta(a|s))^2 (Q(s,a) - b(s))^2] - \underbrace{(\nabla_\theta J(\theta))^2}_{constant} \\
&\sim \arg\min_{b(s)} E_{\pi_\theta}[(\nabla_\theta \log \pi_\theta(a|s))^2] \underbrace{E_{\pi_\theta}[(Q(s,a) - b(s))^2]]}_{\text{least square problem}}
\end{aligned}
$$

- least square solution: $b(s) = E_{\pi_\theta}[Q(s,a)] = V(s)$

# Learning Policy Gradient with Actor-Critic

- The Actor-critic uses the policy gradient method with a critic as a value function baseline and an actor as a policy learner.
- simple pseudocode with TD value evaluation, where we approximate $G_t$ by $G_{t:t+1} = r_t + \gamma V_w(s_{t+1})$.

```
    Critic  Update parameters w of v_w by TD (e.g., one-step) or MC
    Actor   Update θ by policy gradient
function ONE-STEP ACTOR CRITIC
    Initialise s, θ
    for t = 0, 1, 2, ... do
        Sample A_t ~ π_θ(S_t)
        Sample R_{t+1} and S_{t+1}
        δ_t = R_{t+1} + γ v_w(S_{t+1}) − v_w(S_t)        [one-step TD-error, or advantage]
        w ← w + β δ_t ∇_w v_w(S_t)                       [TD(0)]
        θ ← θ + α δ_t ∇_θ log π_θ(A_t | S_t)             [Policy gradient update (ignoring γ^t term)]
```

# Sharing network in Actor-Critic

- Instead of using a separate network for the actor and the critic. We can use a shared network with parameter $\phi$ and a single loss function.
- the mixing loss function for an actor and critic for each trajectory, which minimizes TD loss, maximizes the policy gradient (expected reward) term, and maximizes the entropy regularizer term.

$$L(\phi, \tau) = \lambda_{TD} L_{TD}(s_t, a_t) - \lambda_{PG} J(s_t, a_t) - \lambda H(s_t)$$

- Where

$$\begin{aligned} L_{TD}(s_t, a_t) &= (r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t))^2 \\ J(s_t, a_t) &= (r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)) \log \pi_{\phi(a|s_t)} \\ H(s_t) &= -\sum_a \pi_\phi(a|s_t) \log \pi_\phi(a|s_t) \end{aligned}$$

- Entropy term guarantees the policy remains stochastic; it does not bias towards any action.

# Generalized Advantage Estimation(GAE)

# Bias–Variance Trade-off in Advantage Estimation

- The $n$-step advantage estimate is

$$A_t^{(n)} = G_{t:t+n} - V_w(s_t)$$

- For small $n$, the estimate has **high bias but low variance**. For large $n$ (approaching Monte Carlo), the estimate becomes **low bias but high variance**.

# Bias-Variance Trade Off in Advantage Estimation(continue)

- The **Generalized Advantage Estimator (GAE)** forms an exponentially weighted average of $n$-step advantages:

$$
\begin{aligned}
A_t^{GAE(\gamma,\lambda)} &= (1-\lambda)(A_t^{(1)} + \lambda A_t^{(2)} + \lambda^2 A_t^{(3)} + \ldots) \\
&= (1-\lambda)(\sum_{k=1}^{\infty} \lambda^{k-1} G_{t:t+k} - V(s_t)(\sum_{k=1}^{\infty} \lambda^{k-1} + 1)) \\
&= (1-\lambda) \sum_{k=1}^{\infty} \lambda^{k-1} G_{t:t+k} - V(s_t) \underbrace{((1-\lambda) \sum_{k=1}^{\infty} \lambda^{k-1} + 1 - \lambda)}_{1} \\
&= \underbrace{G_\lambda^t}_{\lambda-return\ method} - V(s_t) = \sum_{i=0}^{\infty} (\gamma\lambda)^i \delta_i \text{ (same with TD}(\lambda)\text{)}
\end{aligned}
$$

- $\lambda - return$ method can be expressed by the discounted sum of TD error with a discount factor.

# GAE with Actor-Critic

- Generalize Advantage Estimator update with Actor-Critic:
  - We implement policy gradient update by backward view from $T-1$ to 0.

$$\text{Recursive formula : } A_t^{GAE(\gamma,\lambda)} = \delta_t + \lambda\gamma A_{t+1}^{GAE(\gamma,\lambda)}$$

$$L(\phi) = \lambda_{TD}(-A_t)^2 - \lambda_{PG} A_t \log \pi_\theta(a|s_t) - \lambda_{ent} H(\pi_\phi(a|s_t))$$

# Natural Policy Gradient Methods

# Stochastic Gradient Descent

- Gradient descent optimization problem:

$$\begin{aligned}\Delta\theta^* &= \arg\min_{\Delta\theta} L_\theta(\theta + \Delta\theta) \\ &= \arg\min_{\Delta\theta} \underbrace{(L_\theta(\theta) + \nabla_\theta L(\theta)\Delta\theta)}_{Taylor\ \ expansion} \\ &\approx \arg\min_{\Delta\theta} \nabla_\theta L(\theta)\Delta\theta\end{aligned}$$

- To guarantee Taylor Expansion with near point approximation, we set a constraint: $||\Delta\theta||_2^2 \leq \epsilon$.
- Transform to an unconstrained optimization problem by adding a Lagrange multiplier:

$$\Delta\theta^* = \arg\min_{\Delta\theta}(\nabla_\theta L(\theta)\Delta\theta + \alpha||\Delta\theta||_2^2)$$

- Solve the Lagrange optimization, we have a solution: $\Delta\theta = \alpha\nabla_\theta L(\theta)$.

# Natural Gradient Descent

- At the previous, we show the Gradient Descent applied in the Stochastic Gradient Descent algorithm.
- SGD is often used to train in ML, but it has a problem with optimizing the probability distribution, such as policy $\pi_\theta(a|s)$.
  - Because, $||\theta_t - \theta_{t+1}||_2^2$ not gaurantee $p_{\theta_t}$ near $p_{\theta_{t+1}}$.
- Natural gradient descent instead of using the KL-divergence:

$$\Delta\theta^* = \arg\min_{\Delta\theta} \nabla_\theta L(\theta)\Delta\theta, s.t. D_{KL}(p_\theta|p_{\theta+\Delta\theta}) \leq \epsilon$$

- Analysis KL-divergence constraint, we can denote $D_{KL}(p_\theta|p_{\theta+\Delta\theta})$ by $D_{KL}(\theta|\theta+\Delta\theta)$. Using Taylor expansion:

$$D_{KL}(\theta, \theta+\Delta\theta) \approx D_{KL}(\theta,\theta) + \nabla_\theta D_{KL}^T(\theta,\theta)\Delta\theta + \frac{1}{2}\Delta\theta^T(\nabla_\theta D_{KL}^T \nabla_\theta D$$

$$= \frac{1}{2}\Delta\theta^T H(\theta)\Delta\theta = \frac{1}{2}\Delta\theta^T F(\theta)\Delta\theta$$

- The Hessian of the KL divergence is the Fisher information matrix.

# Natural Gradient Descent (continue)

- The natural gradient descent optimization becomes:

$$\Delta\theta^* = \arg\min_{\Delta\theta} \nabla_\theta L(\theta)\Delta\theta, \ s.t \ \frac{1}{2}\Delta\theta^T F(\theta)\Delta\theta \leq \epsilon$$

- Lagrange optimization :

$$\Delta\theta^* = \arg\min_{\Delta\theta}(\nabla_\theta L(\theta)\Delta\theta + \alpha\frac{1}{2}\Delta\theta^T F(\theta)\Delta\theta)$$

- Solving the Lagrange optimization, we can find the optimal direction of natural gradient: $\Delta\theta \propto F^{-1}(\theta)\nabla_\theta L(\theta)$.
- Fish Information Matrix: $F(\theta) = E_{p_\theta}[\nabla^2 \log p_\theta]$.
- In practice, we can use the conjugate gradient algorithm to compute $F^{-1}(\theta)$ without having to invert $F(\theta)$.

# Natural actor critic

- Actor-Critic update with natural policy gradient:

$$
\begin{aligned}
\Delta\theta &= \alpha F^{-1}(\theta)\nabla_\theta J(\theta) \\
\nabla_\theta J(\theta) &= A_t^{GAE}\nabla_\theta \log \pi_\theta(a_t|s_t) \\
F(\theta) &= E_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a_t|s_t)\nabla_\theta \log \pi_\theta(a_t|s_t)^T] \\
&\approx \frac{1}{N}\sum_{t=0}^{T}\nabla_\theta \log \pi_\theta(a_t|s_t)\nabla_\theta(a_t|s_t)^T
\end{aligned}
$$

- Natural gradient descent support for policy gradient update is more stable in learning.

# Trust Region Policy Gradient

# Monotonic Improvement Guarantee

- Trust Region Policy Gradient Optimization (TRPO) tries to guarantee a monotonic policy improvement, which satisfies that the performance of policy improvement is a undecrease sequence.

$$J(\pi_0) \leq J(\pi_1) \leq J(\pi_2) \leq ...$$

- Reinforce algorithm is an example of unstable learning, with the large difference between the current and previous policy. TRPO introduces a monotonic guarantee for the stable learning the of policy gradient.

# Monotonic improvemenet theorem

**Theorem (Monotonic Guarantee)**

Let $D_{TV}^{max} = \max_s D_{TV}(\pi(.|s)|\hat{\pi}(.|s))$ be the total variation distance between two policy distribution. And $L_{\pi_{old}}(\pi_{new})$ is the return cumulative expectation of the new policy compute the trajectory generated by the old policy. Then the following bound holds:

$$J(\pi_{new}) \geq L_{\pi_{old}}(\pi_{new}) - \frac{4\epsilon\gamma}{(1-\gamma)^2} D_{TV}^{max}(\pi_{old}, \pi_{new})^2$$

where $\epsilon = \max_{s,a} |A_\pi(s,a)|$

- The proof of this theorem can be found in the TRPO paper.

# Trust region optimization

- The relationship between total variation and KL-divergence distance:

$$D_{TV}(p||q)^2 \leq D_{KL}(p||q)$$

then, $D_{KL}^{max}(\pi, \hat{\pi}) = \max_s D_{KL}(\pi|\hat{\pi}) \geq D_{TV}^{max}(\pi, \hat{\pi})$

- The bound in the monotonic theorem can become:

$$J(\pi_{new}) \geq L_{\pi_{old}}(\pi_{new}) - \frac{4\epsilon\gamma}{(1-\gamma)^2} D_{KL}^{max}(\pi, \hat{\pi})$$

- TRPO algorithm tries to maximize the surrogate objective function:

$$\max_{\theta}[L_{\pi_{old}}(\pi_{new}) - \frac{4\epsilon\gamma}{(1-\gamma)^2} D_{KL}^{max}(\pi, \hat{\pi})]$$

$$= \max_{\theta}[E_{\pi_{new}(a|s)}[\frac{\pi_{old}(a|s)}{\pi_{new}(a|s)} A(s,a)] - \frac{4\epsilon\gamma}{(1-\gamma)^2} D_{KL}^{max}(\pi, \hat{\pi})]$$

- TRPO optimization is similar to the natural gradient descent, with a difference in off-policy importance sampling. Then, using Natural gradient descent for learning TRPO.

# Proximal Policy Optimization

# Clipped Surrogate Objective

- The TRPO algorithm used the conjugate gradient algorithm to deal with a hard constraint objective, which vary complicated to implement.
- PPO expected to provide a simple objective that still satisfies a monotonic improvement, but can apply SGD to learning.
- Clipped surrogate Objective

$$L^{CLIP}(\theta) = E[\min(r_t(\theta)A_t, clip(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t)]$$

  Where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$
- To analyze the clip objective, we consider two cases: $A_t > 0$ and $A_t < 0$.
  - $A_t > 0$:
  - $A_t < 0$:

# PPO Algorithm with GAE

- PPO objective with actor critic and entropy regularization:

$$A_t^{GAE} = \delta_t + \gamma\lambda A_{t+1}^{GAE}$$

$$L(\phi) = \lambda_{TD}(-A_t^{GAE})^2 - \lambda_{PG}L^{CLIP}(\phi) - \lambda_{ent}H(\pi_\phi(a|s))$$

# Off-policy Learning

# Offline Policy

- It is useful to train a policy using data collected from earlier trial agents or parallel workers and stored in a replay buffer [?].
- The learning policy uses the data collected from the distinct policy called offline policy learning.
- The learning policy is called the target policy, while the policy that generated the data is called the behaviour policy.
- For long transitions or trajectory estimate value under difference policy can lead to a distribution shift. The method often used to deal with this problem is importance sampling.

# Evaluation Importance Sampling

- Given a trajectory $\tau_t = \{s_t, a_t, r_{t+1}, s_{t+1}, ...\}$ generated from behaviour policy $\pi_b$.
- we need to estimate the expectation of return value over the target policy $\pi_t$: $E_{s\sim p(s), a\sim\pi_t(a|s)}[G_{\tau_t}]$.
- Importance sampling:

$$E_{s\sim p(s), a\sim\pi_t(a|s)}[G_{\tau_t}] = E_{s\sim p(s), a\sim\pi_b(a|s)}[\frac{p(\tau_t|\pi_t(a|s))}{p(\tau_t|\pi_b(a|s))}G_{\tau_t}]$$

$$= E_{s\sim p(s), a\sim\pi_b(a|s)}[\frac{p(s_t)\pi_t(a_t|s_t)p(s_{t+1})\pi_t(a_{t+1}|s_{t+2})...}{p(s_t)\pi_b(a_t|s_t)p(s_{t+1})\pi_b(a_{t+1|s_{t+2}})...}G_{\tau_t}]$$

$$= E_{s\sim p(s), a\sim\pi_b(a|s)}[\frac{\pi_t(a_t|s_t)\pi_t(a_{t+1}|s_{t+1})...}{\pi_b(a_t|s_t)\pi_b(a_{t+1}|s_{t+1})...}G_{\tau_t}]$$

$$\approx (\prod_{t=0}^{T-1}\frac{\pi_t(a_t|s_t)}{\pi_b(a_t|s_t)})\sum_{t=0}^{T-1}\gamma^t r_t$$

# V-trace

- V-trace is a kind of the importance sampling with truncate value, which extent the R-trace method [?].
- We can reduce the variance of the importance ratio by noting that the reward $r_t$ is independent of the trajectory beyond time $t$. This is called per-decision importance sampling:

$$p_t(\tau) = \prod_{t' \leq t} \frac{\pi(a_{t'}|s_{t'})}{\pi_b(a_{t'}|s_{t'})}$$

- V-trace truncate the IS weight, and used to importance sampling on per-decision

$$c_t = \min(\bar{c}, \frac{\pi(a_t|s_t)}{\pi_b(a_t|s_t)})$$

# V-trace for Actor Critic

- **V-trace critic**
  - The critic evalutation with n-step method can expressed as a sum of temporal error

$$G_{t:t+n} - V(s_t) = \sum_{k=t}^{t+n-1} \gamma^{k-t} \delta_k$$

, The recursive form

$$G_{t:t+n} - V(s_t) = \delta_t + \gamma(G_{t+1:t+n} - V(s_{t+1}))$$

  - By per-decision importance sampling on each TD error at time $t$. We then define the V-trace target value for $G_{t:t+n}$ as

$$G_{t:t+n} = V(s_t) + \sum_{k=t}^{t+n-1} \gamma^{k-t} (\prod_{i \leq k} c_i) \delta_k$$

. The recursive form V-trace for n-step target

$$G_{t:t+n} = V(s_t) + (\prod_{i \leq t} c_i) \delta_t + \gamma(G_{t+1:t+n} - V(s_{t+1})))$$

# V-trace for Actor Critic

- **V-trace actor**
  - As for off-policy gradient objective, we can use a one-step IS ratio

$$\nabla_\theta J_{\pi_b}(\pi_\theta) = E_{s\sim p(s), a\sim\pi_b(a|s)}[\frac{\pi_\theta(a|s)}{\pi_b(a|s)}\nabla_\theta \log \pi_\theta(a|s) Q(s,a)]$$

  - we can replace $Q(s,a)$ by an advanage function with V-trace.
  - note that: the best value for hyper-parameter $\bar{c} = 1$

# Deterministic Policy Gradient

# Deterministic Policy Gradient

**Theorem (Deterministic PG )**

Let $\mu_\theta(s)$ be a deterministic and differentiable policy on the MDP $(S, A, R)$. Then, the deterministic policy gradient is shown below

$$\begin{aligned}\nabla J(\mu_\theta) &= \int_S p_\Theta^\mu(\tau)\nabla_\theta \mu_\theta(s)\nabla_a Q^\mu(s,a)|_{a=\mu_\theta(s)} ds \\ &= E_{\tau\sim p^\mu}[\nabla_\theta \mu_\theta(s)\nabla_a Q^\mu(s,a)|a=\mu_\theta(s)]\end{aligned}$$

Where $\nabla_\theta \mu_\theta(s)$ is the $N_\theta \times N_A$ Jacobian matrix, $N_\theta$ and $N_A$ are the dimension of $\theta$ and $A$.

# Convergence on Deterministic Policy

**Theorem (Limit of Stochastic Policy Gradient)**

Consider a stochastic policy $\pi(a|s)$ such that $\pi_{\theta,\sigma}(a|s) = \mu_\theta(s) + \sigma$, where $\sigma$ is a parameter controlling the variance or noise.

$$\lim_{\sigma \to 0} \nabla_\theta J(\pi_{\theta,\sigma}) = \nabla_\theta J(\mu_\theta)$$

It shows that the deterministic policy gradient is a special case of stochastic with no noise, and can reduce variance.

# Off-policy Deterministic Gradient

- The proof of the two theorems above can be found in paper.
- The deterministic policy does not do any exploration, so we need to use an off-policy method for training.
  - The data collected from a stochastic behaviour policy $\pi_b$.
  - The objective is just to integrate on the state distribution, then we do not use importance sampling for off-policy deterministic.
  - We approximate $Q^\mu(s,a)$ by an approximation function $Q^w(s,a)$ using TD evaluation method.

$$\delta = r_t + \gamma Q_w(s_{t+1}, \mu_\theta(s_{t+1})) - Q_w(s_t, a_t)$$
$$w_{t+1} \leftarrow w_t + \alpha\delta\nabla_w Q_w(s_t, a_t)$$
$$\theta_{t+1} \leftarrow \theta_t + \alpha\nabla_\theta \mu_\theta(s_t)\nabla_a Q_w(s_t, a)$$

  - If $\nabla_a Q_w(s,a) = (\nabla_\theta \mu_\theta)^T w$ is linear in $W$, and $W$ minimize the mean squared error of approximation. Then, the function approximation $Q_w(s,a)$ is called compatible with the actor, and it does not bias the overall gradient.

# Deep Deterministic Policy Gradient (DDPG) Algorithm

**Algorithm 1** DDPG algorithm

Randomly initialize critic network $Q(s, a|\theta^Q)$ and actor $\mu(s|\theta^\mu)$ with weights $\theta^Q$ and $\theta^\mu$.
Initialize target network $Q'$ and $\mu'$ with weights $\theta^{Q'} \leftarrow \theta^Q$, $\theta^{\mu'} \leftarrow \theta^\mu$
Initialize replay buffer $R$
**for** episode = 1, M **do**
  Initialize a random process $\mathcal{N}$ for action exploration
  Receive initial observation state $s_1$
  **for** t = 1, T **do**
    Select action $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$ according to the current policy and exploration noise
    Execute action $a_t$ and observe reward $r_t$ and observe new state $s_{t+1}$
    Store transition $(s_t, a_t, r_t, s_{t+1})$ in $R$
    Sample a random minibatch of $N$ transitions $(s_i, a_i, r_i, s_{i+1})$ from $R$
    Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q'})$
    Update critic by minimizing the loss: $L = \frac{1}{N}\sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$
    Update the actor policy using the sampled policy gradient:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N}\sum_i \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu}\mu(s|\theta^\mu)|_{s_i}$$

    Update the target networks:

$$\theta^{Q'} \leftarrow \tau\theta^Q + (1-\tau)\theta^{Q'}$$
$$\theta^{\mu'} \leftarrow \tau\theta^\mu + (1-\tau)\theta^{\mu'}$$

  **end for**
**end for**

# Twin Delayed DPPG (TD3)

- TD3 provides several changes from DDPG to address with Q-learning overestimates in actor-critic and variance error propagation.
- Clipped Q-learning avoids overestimating by taking the minimum between two critics

$$y = r + \gamma min_{i=1,2} Q_{\theta_i'}(s, a)$$

- Delayed policy updates critic every step, but it just update actor when the critic is stable to deal with error accumulation.
- Target policy smoothing: add a noise in the deterministic policy to encourage generalization

$$a = \pi_\theta(s) + \epsilon,\ \epsilon \sim clip(N(0, \sigma), -c, c)$$

# TD3 Pseudocode

**Algorithm 1** TD3

Initialize critic networks $Q_{\theta_1}$, $Q_{\theta_2}$, and actor network $\pi_\phi$
with random parameters $\theta_1$, $\theta_2$, $\phi$
Initialize target networks $\theta_1' \leftarrow \theta_1$, $\theta_2' \leftarrow \theta_2$, $\phi' \leftarrow \phi$
Initialize replay buffer $\mathcal{B}$
**for** $t = 1$ **to** $T$ **do**
  Select action with exploration noise $a \sim \pi_\phi(s) + \epsilon$,
  $\epsilon \sim \mathcal{N}(0, \sigma)$ and observe reward $r$ and new state $s'$
  Store transition tuple $(s, a, r, s')$ in $\mathcal{B}$

  Sample mini-batch of $N$ transitions $(s, a, r, s')$ from $\mathcal{B}$
  $\tilde{a} \leftarrow \pi_{\phi'}(s') + \epsilon, \quad \epsilon \sim \text{clip}(\mathcal{N}(0, \tilde{\sigma}), -c, c)$
  $y \leftarrow r + \gamma \min_{i=1,2} Q_{\theta_i'}(s', \tilde{a})$
  Update critics $\theta_i \leftarrow \text{argmin}_{\theta_i} N^{-1} \sum (y - Q_{\theta_i}(s, a))^2$
  **if** $t$ mod $d$ **then**
    Update $\phi$ by the deterministic policy gradient:
    $\nabla_\phi J(\phi) = N^{-1} \sum \nabla_a Q_{\theta_1}(s, a)|_{a=\pi_\phi(s)} \nabla_\phi \pi_\phi(s)$
    Update target networks:
    $\theta_i' \leftarrow \tau\theta_i + (1 - \tau)\theta_i'$
    $\phi' \leftarrow \tau\phi + (1 - \tau)\phi'$
  **end if**
**end for**

# Reinforcement Learning as Probabilistic Inference

# from RL to Probabilistic Inference

- The original policy gradient objective wants to find an optimal policy $\pi_\theta(a|s)$ that maximizes the cumulative expected value over the trajectory distribution

$$\theta^* = \arg\max \sum_{t=1}^{T} E_{(s_t,a_t)\sim p(s_t,a_t|\theta)}[r(s_t,a_t)]$$

- From the decision-making problem to the inference problem, we must formulate a probabilistic graphical model so that "the most probable trajectory " corresponds to the trajectory from the optimal policy.
- Motivation for formulating RL as an inference problem:
  - Effective tool for inverse RL.
  - Exploitment of several powerful approximate inference algorithms in RL learning.
  - provides a natural exploration based on entropy maximization like soft Q-learning.

# RL Graphical Model

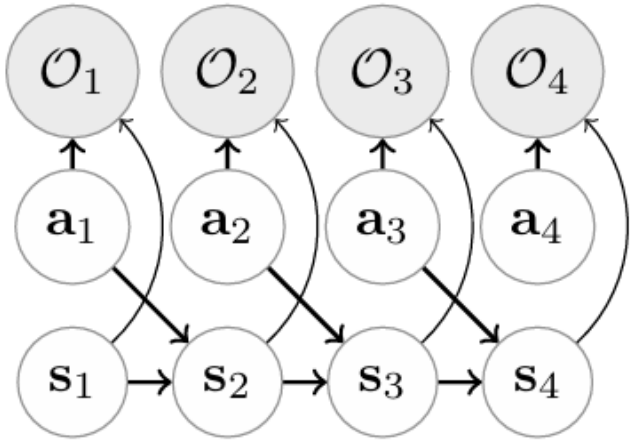


Figure: Graphical model with $s$ state variable, $a$ action variable, $o$ optimality variable.

- The graphical model above represents three factors: $p(s_1)$(initial state ), $p(s_{t+1}|s_t, a_t)$ (transition state), $p(o_t|s_t, a_t)$ (optimal time step if $o_t = 1$ else $o_t = 0$)
- we model the optimal time step by an energy-based reward:

$$p(o_t = 1|s_t, a_t) = \exp(\frac{1}{\alpha} r(s_t, a_t))$$

# RL Graphical Model (continue)

- The trajectory distribution under the optimality time step observation from this graphical model:

$$
\begin{aligned}
p(\tau|o_{1:T}) &= p(s_1)\prod_{t=1}^{T} p(s_{t+1}|s_t,a_t)p(o_t=1|s_t,a_t) \\
&= p(s_1)\prod_{t=1}^{T} p(s_{t+1}|s_t,a_t)\exp(r(s_t,a_t)) \\
&= [p(s_1)\prod_{t=1}^{T} p(s_{t+1}|s_t,a_t)]\exp(\sum_{t=1}^{T} r(s_t,a_t))
\end{aligned}
$$

- From this probability, the trajectory with the highest reward has the highest probability, and otherwise.
- the goal is to find a posteriori optimal policy with optimal observation $\pi(a_t|s_t,o_t)$.

# Deterministic Case Inference

- we can set a goal with a more achievable trajectory when conditioned on the optimal variable.

$$\arg\max_{\tau} p(\tau|o=1,\pi)$$

$$\arg\max_{\tau}[p(s_1|o=1,\pi)\prod_{t=1}^{T}\pi(a_t|s_t,o_t)p(s_{t+1}|s_t,a_t,o_t)][\prod_{t=1}^{T}p(o_t=1|s_t,a$$

$$\arg\max_{\pi} p(s_1)\exp(\sum_{t=1}^{T}r(s_t,a_t))$$

- We consider inference in the case of an undirected graphical model. This process consists of planning an open-loop policy and evaluating it for each initial state.

# Stochastic Case Inference

- In the stochastic case, it is hard to deal with inference conditional in optimal variable. Conditional optimal variable on $p(s_{t+1}|s_t, a_t)$ and $p(s_1)$ leads to a problem with optimistic bias, which is discussed in the original paper [?].
- The dynamic probability and initial state are fixed, and using variational inference.
- The objective is to maximize the log likelihood optimal variable to learn a policy that generates a distribution over optimal trajectories.

# Variational Inference in Stochastic Case

- we introduce a variational distribution $q(\tau)$ to replace $p(\tau)$. With $q(\tau)$ keep the same $p(s_1)$ and $p(s_{t+1}|s_t, a_t)$ with $p(\tau)$

$$q(\tau) = p(s_1) \prod_{t=1}^{T} p(s_{t+1}|s_t, a_t) q(s_t|a_t)$$

- The variational lower bound or ELBO

$$\begin{aligned}
\log p(o=1) &= \log \int p(o=1,\tau) d_\tau \\
&= log \int p(o=1,\tau) \frac{q(\tau)}{q(\tau)} d_\tau \\
&= log E_{\tau \sim q(\tau)} [\frac{p(o=1,\tau)}{q(\tau)}] \\
&\underbrace{\geq}_{jensen' sinequality} E_{\tau \sim q(\tau)} [\log \frac{p(o=1,\tau)}{q(\tau)}] \\
&= E_{\tau \sim q(\tau)} [\log p(o=1|\tau)] - KL(q(\tau)|p(\tau))
\end{aligned}$$

# ELBO Objective

- The objective of the policy learning task is to maximize the ELBO. And the constraint on the probability range of the policy

$$
\begin{aligned}
J(\pi_q, \pi_p) &= E_{\tau \sim q(\tau)}[\log \exp(\frac{1}{\alpha} G(a_t, s_t))] - KL(\pi_q(a_t|s_t)|\pi_p(a_t|s_t)) \\
&= \sum_{t=1}^{T} E_{\tau \sim q(\tau)}[\frac{1}{\alpha} G(a_t, s_t) - KL(\pi_q(a_t|s_t)|\pi_p(a_t|s_t))]
\end{aligned}
$$

$$
s.t \int d_{q(s)} \int \pi_p(a_t|s_t) da ds = 1, \int d_{p(s)} \int \pi_q(a_t|s_t) da ds = 1
$$

- The objective represents an undiscounted reward; we can add a discount factor to guarantee convergence, like normal reinforcement learning.
- The problem have solved with two main ways including EM control such as MPO algorithm, and KL control such as Soft actor critic.

# EM Control(1)

**EM (Expectation-Maximization) algorithm**

- EM is often used to deal with ML inference or MAP inference with a latent variable.
- let $X$ be the observation variable, $Z$ be latent variable. We have a crash introduction of EM algorithm for ML inference

$$\max_{\theta} \log p(x|\theta)$$
$$\max_{\theta} \log \int q(z) \frac{p(z, x|\theta)}{q(z)} dz$$
$$\leq \max_{\theta} E_{q(z|)}[\log p(z, x|\theta)] - E_{q(z)}[\log q(z)]$$

- **E-step** fix the parameter $\theta$, the optimal solution is $q(z) = q(z|x)$
- **M-step** fix the $\phi$ and optimize to update new $\theta$

# EM Control (2)

- **E-step** at iteration k, we force the variational posterior policy $\pi_q^{(k)}$ to be close to the optimal posterior policy, the maximizer of the ELBO $J(\pi_{\theta_p}^{(k-1)}, \pi_q)$ with respect to non-parametric $\pi_q$. EM control converts the hard-constraint optimization problem to solving the following Lagrangian.

$$
\begin{aligned}
J(\pi_q, \beta) = & \int d_\pi(s) \int \pi_q(a|s) \alpha^{-1} G(s,a) da ds \\
& - \int d_\pi(s) \int \pi_q(a|s) \log \frac{\pi_q(a|s)}{\pi_{\theta_p}^{(k-1)}(a|s)} da ds \\
& + \beta(1 - \int d_\pi(s) \int \pi_q(a|s) da ds)
\end{aligned}
$$

We solve the above Lagrangian problem and obtain the solution:

$$
\pi_q^{(k)}(a|s) = \pi_{\theta_p}^{(k-1)} Z(s)^{-1} \exp(\alpha^{-1} G(s,a))
$$

Where $Z(s)$ is the partition function.

# EM Control(3)

- **M-step** at iteration k, we maximize the ELBO $J(\pi_q^{(k)}, \pi_{\theta_p})$ with respect to $\pi_p$. Considering the optimization with respect to $\pi_p$, we remove all factors that do not depend on $\pi_p$.

$$\max_p E_{d_\pi(s)\pi_q^{(k)}(a|s)}[\log \pi_p(a|s)] = \max_p E_{d_\pi(s)\pi_{\theta_p}^{(k-1)}}[\frac{\log \pi_{\theta_p}(a|s)}{Z(s)} \exp(r(s,a))]$$

- where we may approximate $Z(s) \approx \frac{1}{M}\sum_{j=1}^{M} \exp(\alpha^{-1}G(s,a))$ with $a_j \sim \pi_{\theta_p}^{(k-1)}(a|s)$ in practice.
- The temperature can be obtained by optimizing a dual function, as described in detail in the MPO algorithm section.

# KL Control

- In KL control, we only optimize the variational posterior $\pi_q$, holding the prior $\pi_p$ fixed like the E-step in EM control.
- In addition, we represent $\pi_q$ parameterically as $\pi_{\theta_p}$ instead of the non-parametric approach used by EM.

$$J(\pi_{\theta_q}, \pi_p) = \max_{\theta_q} E_{d_\pi(s)\pi_{\theta_q}(a|s)}[\alpha^{-1}G(s,a) - \log \frac{\pi_{\theta_q}(a|s)}{\pi_p(a|s)}]$$

- If $\pi_p$ is uniform, we have a form of the objective called the maximum entropy RL objective.

$$\alpha J(\pi_{\theta_q}, \pi_p) = \max_{\theta_q} E_{d_\pi(s)\pi_{\theta_q}(a|s)}[r(s,a) + \alpha H(\pi(a|s))]$$

- The way to find the temperature $\alpha$ is also discussed in the SAC algorithm section.

# Maximum a Posteriori Policy Optimisation(MPO)

# Policy Improvement

- MPO is an instance of EM control mentioned in the previous section.
- The objective of the learning task of MPO is the same as that of EM control, except for the discounted factor in reward:

$$J(q,\theta) = \int \mu_q(s) \int q(s|a)Q(s,a)dads + \log p(\theta_p^{(k-1)})$$
$$s.t \int d_\pi(s) \int \pi_q(a|s)dads = 1$$
$$\int \mu_q(s) KL(q(s|a), \pi_{\theta_p^{(k-1)}}(a|s))ds \leq \epsilon$$

- $\log p(\theta_p^{(k-1)})$ is a prior regularization term that keeps the updated policy close to the prior during the M-step.

# Policy Improvement

- **E-step**: The original paper MPO introduces two versions of E-step with non-parametric and parametric variational distribution $q(a|s_t)$. However, we only consider the nonparametric variational distribution $q(a|s_t)$ when optimizing the objective function.
- Solve the constraint objective by the Lagrangian method, like EM control, we have an optimal non-parametric form of $q(a|s_t)$

$$q^k(a|s) = \frac{1}{Z(s)} \pi_{\theta_p^{(k-1)}} \exp(\alpha^{-1} Q(s,a))$$

$$Z(s) = E_{\pi_{\theta_p^{(k-1)}}}[\exp(\alpha^{-1} Q(s,a)] \approx \frac{1}{M} \sum_{j=1}^{M} \exp(\alpha^{-1} Q(s,a_j))$$

$$a_j \sim \pi_{\theta_p^{(k-1)}}(a|s)$$

- The temperature $\alpha$ is solved for by minimizing the dual function

$$G(\alpha) = \alpha\epsilon + \alpha \log E_{d(s),\pi_{\theta_p^{(k-1)}}}[\exp(\alpha^{-1} Q(s,a))]$$

# Policy Improvement

- **M-step** update policy prameter by optimize the objective respect to $\pi_{\theta_p^{(k-1)}}$ and a soft regularization prior distribution $p(\theta_p^{(k-1)})$ in the objective.
- we assume the prior is a Gaussian centered at the previous parameters.

$$p(\theta_p^{(k-1)}) \sim N(\mu = \theta_p^{(k-1)} | \Sigma = \frac{F_{\theta_p^{(k-1)}}}{\lambda}) = c \exp(-\lambda(\theta_p^k - \theta_p^{(k-1)})^T F^{-1}_{\theta_p^{(k-}}$$

- The term $\lambda(\theta_p^k - \theta_p^{(k-1)})^T F^{-1}_{\theta_p^{(k-1)}}(\theta - \theta_p^{(k-1)})$ í the second order taylor approximation of $\int \mu_q(s) KL(\pi(a|s, \theta_q^{(k-1)}) | \pi(a|s, \theta_q^k)) ds$ which leads us to the generalised M-step objective:

$$\max_{\pi} E_{\mu_q(s)}[E_{q(a|s)}[\log \pi(a|s, \theta)] - \lambda KL(\pi(a|s, \theta_p^{(k-1)}) | \pi(a|s, \theta))]$$

- we can then optimize this objective using SGD to find the new update $\theta_p^{(k-1)}$ for the policy.

# Policy Evaluation

- MPO is an off-policy actor-critic. It employed the V-trace actor-critic to stabilize in policy evaluation

$$\min_{\phi} L(\phi) = \min_{\phi} E_{\mu_b(s),\pi_b(a|s)}[(Q_\phi(s,a) - Q_t^{ret})^2]$$

$$Q_t^{ret} = Q_{\phi'}(s_t, a_t) + \sum_{j=t}^{\infty} \gamma^{j-t} (\prod_{k=t+1}^{j} c_k)[r(s_j, a_j) + E_{\pi(a|s_{j+1})}[Q_{\phi'}(s_{j+1},$$

$$c_k = \min(1, \frac{\pi(a_k|s_k)}{\pi_b(a_k|s_k)})$$

# MPO Pseudocode

**Algorithm 1** MPO (chief)

```
Input G number of gradients to average
while True do
    initialize N = 0
    initialize gradient store s_φ = {}, s_η = {}, s_{η_μ} = {}, s_{η_Σ} = {} s_θ = {}
    while N < G do
        receive next gradient from worker w
        s_φ = s_φ + [δφ^w]
        s_φ = s_θ + [δθ^w]
        s_η = s_η + [δη^w]
        s_{η_μ} = s_{η_μ} + [δη_μ^w]
        s_{η_θ} = s_{η_θ} + [δη_θ^w]
    update parameters with average gradient from
    s_φ, s_η, s_{η_μ}, s_{η_Σ} s_θ
    send new parameters to workers
```

**Algorithm 2** MPO (worker) - Non parametric variational distribution

```
Input = ϵ, ϵ_Σ, ϵ_μ, L_max
i = 0, L_curr = 0
Initialise Q_{ω_i}(a, s), π(a|s, θ_i), η, η_μ, η_Σ
for each worker do
    while L_curr > L_max do
        update replay buffer B with L trajectories from the environment
        k = 0
        // Find better policy by gradient descent
        while k < 1000 do
            sample a mini-batch B of N (s, a, r) pairs from replay
            sample M additional actions for each state from B, π(a|s, θ_i) for estimating integrals
            compute gradients, estimating integrals using samples
            // Q-function gradient:
            δ_φ = ∂_φ L'_φ(φ)
            // E-Step gradient:
            δη = ∂_η g(η)
            Let: q(a|s) ∝ π(a|s, θ_i) exp(Q_{θ_t}(a,s,φ') / η)
            // M-Step gradient:
            [δ_{η_μ}, δ_{η_Σ}] = α∂_{η_μ,η_Σ} L(θ_k, η_μ, η_Σ)
            δ_θ = ∂_θ L(θ, η_{μ_{k+1}}, η_{Σ_{k+1}})
            send gradients to chief worker
            wait for gradient update by chief
            fetch new parameters φ, θ, η, η_μ, η_Σ
            k = k + 1
        i = i + 1, L_curr = L_curr + L
        θ_i = θ, φ' = φ
```

# Soft Actor-Critic

# Maximum Entropy Reinforcement Learning

- SAC is a kind of maximum entropy reinforcement learning, with an objective in the form of KL control discussed in the previous section.

$$\pi^* = \arg\max \sum_t^T E_{(s_t,a_t)\sim p_\pi}[r(s_t,a_t) + \alpha H(\pi(a|s_t))]$$

- The term entropy is a regularization on the policy at each time step, whose purpose is to control the stochasticity and exploration of the policy.
- Maximum entropy RL derived a soft policy iteration backup, which proved monotonic convergence like Bellman backup in the conventional reinforcement learning.

# Soft Q-learning Iteration (1)

**Definition (soft bellman backup)**

Let $Q : S \times A \to R$, and the soft bellman backup $T^\pi : Q_{old} \to Q_{new}$ be defined as

$$T^\pi Q(s_t, a_t) = r(s_t, a_t) + \gamma E_{s_{t+1} \sim p}[V(s_{t+1})]$$

, where $V(s_t)$ is the soft value function

$$V(s_t) = E_{a_t \sim \pi}[Q(s_t, a_t) - \alpha \log \pi(a_t | s_t)]$$

- The soft Bellman backup is a foundation tabular case to derive the soft actor-critic.
- The operator soft bellman backup $T^\pi : Q_{old} \to Q_{new}$ proofed monotonic convergence in the next slide.

# Soft Q-learning Iteration (2)

- The soft policy iteration theorem can be partitioned into two lemmas, including soft policy evaluation and soft policy improvement.

**lamma 1 (Soft Policy Evaluation)**

Consider the soft bellman backup operator $T^{\pi}$ in the previous definition and a mapping $Q : S \times A \to R$ with $|A| < \infty$ and define $Q^{k+1} = T^{\pi} Q^{k}$. Then the sequence $Q^{k}$ will converge to the soft Q-function of $\pi$ as $k \to \infty$.

**lamma 2 (Soft Policy Improvement)**

Let $\prod$ be an assumption set policy, and when the optimizer of the minimization problem $(*)$, then $Q^{\pi_{new}}(s_t, a_t) \geq Q^{\pi_{old}}(s_t, a_t)$ for all $(s_t, a_t) \int S \times A$ with $|A| < \infty$.

$$\pi_{new} = \arg \min_{\pi' \in \prod} D_{KL}(\pi'(a|s_t)|\frac{\exp(\frac{1}{\alpha} Q^{\pi_{old}}(s_t, a_t))}{Z^{old}(s_t)})(*)$$

# Soft Q-learning Iteration (3)

**Theorem (Policy Iteration)**

Repeated application of soft policy evaluation and soft policy improvement from any $\pi \in \prod$ converges to a policy $\pi^*$ such that $Q^{\pi^*}(s_t, a_t) \geq Q^{\pi}(s_t, a_t)$ for all $\pi \in \prod$ and $(s_t, a_t) \in S \times A$, asuming $|A| < \infty$.

- All proofs of the two lemmas and the theorem detail in the SAC paper [?].

# Soft Q-learning Iteration(4)

- we now derive the optimal policy that guarantees policy improvement in the Lamma 2 by maximizing the value function.

$$V^*(s) = \max_a \sum_a \pi(a|s)[Q^*(s,a) - \alpha \log \pi(a|s)]$$
$$= \min_a \sum_a \pi(a|s)[\alpha \log \pi(a|s) - Q^*(s,a)]$$

- This is a contrained optimization problem where $\pi(a|s)$ is a probability distribution. Then, the Lagrange form of the optimization problem:

$$L(\pi, \lambda) = \sum_a \pi(a|s)[\alpha \log \pi(a|s) - Q^*(s,a)] + \lambda(1 - \sum_a \pi(a|s))$$

- Taking the derivative of $L(\pi, \lambda)$ respect to $\pi(s|a)$ and solve the equation $\frac{\partial L(\pi,\lambda)}{\partial \pi(a|s)} = 0$. We obtain a solution:

$$\log \pi(a|s) = \frac{Q^*(s,a) + \lambda - \alpha}{\alpha}$$

# Soft Q-learing Iteration (5)

- From the solution obtained, we can derive the optimal policy

$$\pi^*(a|s) \propto \exp(\frac{Q^*(s,a)}{\alpha})$$
$$\pi(a|s) = \frac{\exp(Q^*(s,a)/\alpha)}{\sum_{a'} \exp(Q^*(s,a)/\alpha)}$$

- replace optimal $\pi^*(a|s)$ we obtain an optimal value function

$$V^*(s) = \alpha \sum_a \exp \frac{Q^*(s,a)}{\alpha}$$

# Soft Actor Critic

- From the soft Q-learning method, we derive it to soft actor critic with parameterization.
- SAC have two seraparate networks $\theta$ for policy $\pi_\theta(a|s)$ and $w$ for soft Q-value $Q_w(s,a)$.
- **SAC policy evaluation**: fixed $\pi_\theta(a|s)$ and optimize the parameters $Q_w(s,a)$ by minimizing

$$J_Q(w) = E_{(s_t,a_t)\sim D}[\frac{1}{2}(Q_w(s_t,a_t) - (r(s_t,a_t) + \gamma E_{s_{t+1}\sim p}[V_{\bar{w}}(s_{t+1})]))^2]$$

  And the soft value function $V_{\bar{w}}(s_t)$

$$V_{\bar{w}}(s_{t+1}) = E_{\pi(a_t|s_{t+1})}[Q_{\bar{w}}(s_{t+1},a_{t+1}) - \alpha \log \pi_\theta(a_{t+1}|s_{t+1})]$$

- To avoid the positive overestimation bias occured with actor critic. SAC used to the clipped double Q learning like TD3

$$y_t = r(s_t,a_t) + \gamma[\min_{i=1,N} Q_{\bar{w}_i}(s_{t+1},a_{t+1}) - \alpha \log \pi_\theta(a_{t+1}|s_{t+1})]$$

# Soft Actor Critic

- final step in policy evaluation, we take the gradient repecst to $w$

$$\hat{\nabla}_w J_Q(w) = \nabla_w Q_w(s_t, a_t)(Q_w(s_t, a_t) - y_t)$$

- Note that the $\bar{w}$ is exponential moving average (EMA) of $w$, which has a form $\bar{w} = (1-p)\bar{w} + pw$. Especially, $\bar{w}$ target weight of critic used to compute TD error showed in th previous slide. Instead, $w$ is the online weight training.
- **SAC policy improvement** we now generalize soft policy improvement in lammas 2 to non-tabular case. we fixed $w$ and optimize the parameters $\theta$ of the policy by minimizing the objective $(*)$ show in lammas 2.

$$J_\pi(\theta) = E_{s\sim D}[E_{a_t\sim\pi}[\alpha \log \pi_\theta(a_t|s_t) - Q_w(s_t, a_t)]]$$

# Soft Actor Critic

- To optimize the objective, we often use REINFORCE in the policy gradient. This method just take gradient respect to $\theta$ network which can backpropagate. However, this method often has high variance problem.
- SAC use a reparameterization trick which take gradient on both action $a_t$ and network $\theta$.

$$\pi_\theta(a_t|s_t) = N(\mu_\theta(s_t), \sigma_\theta^2(s_t))$$

- However, when we sample action this sampling operator can not take gradient, then we use a reparameterization trick to do it.

$$a \sim f_\theta(s_t, \epsilon_t)$$

$$J_\pi(\theta) = E_{s_t \sim D, \epsilon_t \sim N}[\alpha \log \pi_\theta(f_\theta(\epsilon_t, s_t)|s_t) - Q_w(s_t, f_\theta(\epsilon_t, s_t))]$$

- The gradient of the reparameterization objective

$$\begin{aligned}\hat{\nabla}_\theta J_\pi(\theta) &= \nabla_\theta \alpha \log(\pi_\theta(a_t|s_t)) \\ &\quad + (\nabla_{a_t} \alpha \log(\pi_\theta(a_t|s_t)) - \nabla_{a_t} Q(s_t, a_t)) \nabla_\theta f_\theta(\epsilon_t, a_t)\end{aligned}$$

# Soft Actor Critic

- **Adjusting the temperature** SAC propose to automatically adjust the temperature parameter $\alpha$ by optimizing

$$J(\alpha) = E_{s_t \sim D, a_t \sim \pi_\theta}[-\alpha(\log \pi_\theta(a_t|s_t) + \bar{H})]$$

- where $\bar{H}$ is a threshold value on the entropy of the policy (a hyper-parameter).

# SAC Pseudocode

**Algorithm 11:** SAC

Initialize environment state $\boldsymbol{s}$, policy parameters $\boldsymbol{\theta}$, $N$ critic parameters $\boldsymbol{w}_i$, target parameters $\overline{\boldsymbol{w}}_i = \boldsymbol{w}_i$, replay buffer $\mathcal{D} = \emptyset$, discount factor $\gamma$, EMA rate $\rho$, step size $\eta_w$, $\eta_\pi$.
**repeat**
  Take action $\boldsymbol{a} \sim \pi_{\boldsymbol{\theta}}(\cdot|\boldsymbol{s})$
  $(\boldsymbol{s}', r) = \text{step}(\boldsymbol{a}, \boldsymbol{s})$
  $\mathcal{D} := \mathcal{D} \cup \{(\boldsymbol{s}, \boldsymbol{a}, r, \boldsymbol{s}')\}$
  $\boldsymbol{s} \leftarrow \boldsymbol{s}'$
  **for** $G$ *updates* **do**
    Sample a minibatch $\mathcal{B} = \{(\boldsymbol{s}_j, \boldsymbol{a}_j, r_j, \boldsymbol{s}'_j)\}$ from $\mathcal{D}$
    $\boldsymbol{w} = \text{update-critics}(\boldsymbol{\theta}, \boldsymbol{w}, \mathcal{B})$
  Sample a minibatch $\mathcal{B} = \{(\boldsymbol{s}_j, \boldsymbol{a}_j, r_j, \boldsymbol{s}'_j)\}$ from $\mathcal{D}$
  $\boldsymbol{\theta} = \text{update-policy}(\boldsymbol{\theta}, \boldsymbol{w}, \mathcal{B})$
**until** *converged*;
.
def update-critics$(\boldsymbol{\theta}, \boldsymbol{w}, \mathcal{B})$:
Let $(\boldsymbol{s}_j, \boldsymbol{a}_j, r_j, \boldsymbol{s}'_j)_{j=1}^{B} = \mathcal{B}$
$y_j = y(r_j, \boldsymbol{s}'_j; \overline{\boldsymbol{w}}_{1:N}, \boldsymbol{\theta})$ for $j = 1 : B$
**for** $i = 1 : N$ **do**
  $\mathcal{L}(\boldsymbol{w}_i) = \frac{1}{|\mathcal{B}|} \sum_{(\boldsymbol{s},\boldsymbol{a},r,\boldsymbol{s}')_j \in \mathcal{B}} (Q_{\boldsymbol{w}_i}(\boldsymbol{s}_j, \boldsymbol{a}_j) - \text{sg}(y_j))^2$
  $\boldsymbol{w}_i \leftarrow \boldsymbol{w}_i - \eta_{\boldsymbol{w}} \nabla \mathcal{L}(\boldsymbol{w}_i)$ // Descent
  $\overline{\boldsymbol{w}}_i := \rho \overline{\boldsymbol{w}}_i + (1-\rho) \boldsymbol{w}_i$ //Update target networks
Return $\boldsymbol{w}_{1:N}, \overline{\boldsymbol{w}}_{1:N}$
.
def update-actor$(\boldsymbol{\theta}, \boldsymbol{w}, \mathcal{B})$:
$\hat{Q}(s, a) \triangleq \frac{1}{N} \sum_{i=1}^{N} Q_{\boldsymbol{w}_i}(s, a)$ // Average critic
$J(\boldsymbol{\theta}) = \frac{1}{|\mathcal{B}|} \sum_{\boldsymbol{s} \in \mathcal{B}} \left( \hat{Q}(\boldsymbol{s}, \tilde{\boldsymbol{a}}_{\boldsymbol{\theta}}(\boldsymbol{s})) - \alpha \log \pi_{\boldsymbol{\theta}}(\tilde{\boldsymbol{a}}(\boldsymbol{s})|\boldsymbol{s}) \right)$, $\tilde{\boldsymbol{a}}_{\boldsymbol{\theta}}(\boldsymbol{s}) \sim \pi_{\boldsymbol{\theta}}(\cdot|\boldsymbol{s})$
$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \eta_{\boldsymbol{\theta}} \nabla J(\boldsymbol{\theta})$ // Ascent
Return $\boldsymbol{\theta}$

# Reference

# References I


1. R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018.
2. K. P. Murphy, *Reinforcement Learning: An Overview*, Technical Report / Tutorial Notes.
3. D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller,*Deterministic Policy Gradient Algorithms*, Proceedings of ICML, 2014.
4. J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, *Proximal Policy Optimization Algorithms*, 2017.
5. J. Schulman, P. Moritz, S. Levine, M. I. Jordan, and P. Abbeel, *High-Dimensional Continuous Control Using Generalized Advantage Estimation*, ICLR, 2016.
6. T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra,*Continuous Control with Deep Reinforcement Learning*, ICLR, 2016.

# References II


7 J. Schulman, S. Levine, P. Moritz, M. I. Jordan, and P. Abbeel, *Trust Region Policy Optimization*, Proceedings of ICML, 2015.

8 S. Fujimoto, H. van Hoof, and D. Meger, *Addressing Function Approximation Error in Actor-Critic Methods*, International Conference on Machine Learning (ICML), 2018.

9 R. Munos, A. Harutyunyan, T. Stepleton, and M. G. Bellemare, *Safe and Efficient Off-Policy Reinforcement Learning*, NeurIPS, 2016.

10 L. Espeholt, H. Soyer, R. Munos, K. Simonyan, V. Mnih, T. Ward, Y. Doron, V. Firoiu, T. Harley, I. Dunning, S. Legg, and K. Kavukcuoglu, *IMPALA: Scalable Distributed Deep-RL with Importance Weighted Actor-Learner Architectures*, ICML, 2018.

11 H. Furuta, T. Matsushima, T. Kozuno, Y. Matsuo, and S. S. Gu, *Co-Adaptation of Algorithmic and Implementational Innovations in Inference-based Deep Reinforcement Learning*, NeurIPS, 2021.

# References III


12 S. Levine, *Reinforcement Learning and Control as Probabilistic Inference: Tutorial and Review*, arXiv:1805.00909, 2018.

13 A. Abdolmaleki, J. T. Springenberg, Y. Tassa, R. Munos, N. Heess, and M. Riedmiller,*Maximum a Posteriori Policy Optimisation*, International Conference on Learning Representations (ICLR), 2018.

14 T. Haarnoja, S. Ha, A. Zhou, J. Tan, K. Hartikainen, V. Kumar, H. Zhu, P. Abbeel, and S. Levine, *Soft Actor-Critic Algorithms and Applications*, arXiv:1812.05905, 2019.

Thank you for watching !!!